WORLD-CLASS XR / MOBILE SYSTEMS ENGINEERING · UX/UI DESIGN · AI ORCHESTRATION

AI AGENTIC WEB APP BUILD PLAYBOOK

A complete end-to-end framework for building an AI-powered creator brand using a 6-agent agentic pipeline system. Any creator. Any brand. Any goal. This guide uses **Metakittyz** as the working example — swap in your brand anywhere you see `[VARIABLE]` markers.

Example Brand: METAKITTYZ

Goal: 1M Followers in 90 Days

6-Agent AI Pipeline

Beginner-Friendly · No Prior Coding Required

Version 1.0 · 2025

METAKITTYZ

AI Agentic Build Playbook · Template Framework

MATRIX → NOVA → SPARK → PULSE → ARIA → GUARDIAN

Built Live on Stream · erika@productimagination.com

Table of Contents

00 TEMPLATE GUIDE

How to Use This as a Template

This playbook is a reusable framework. Replace every `[VARIABLE]` with your brand specifics. The system architecture, agent pipeline, and build process are universal — only the brand content changes.

TEMPLATE VARIABLES — REPLACE THESE 10 VALUES EVERYWHERE

Find every `[VARIABLE]` in this document and substitute your own values. The logic stays identical — your brand slots in.

VARIABLE	METAKITTYZ VALUE (EXAMPLE)	YOUR BRAND VALUE
`[BRAND_NAME]`	METAKITTYZ	_____
`[BRAND_DOMAIN]`	metakittyz.com	_____
`[BRAND_EMAIL]`	erika@productimagination.com	_____
`[GITHUB_USER]`	erikaherreraxr	_____
`[GOAL]`	1,000,000 followers in 90 days	_____
`[PLATFORM]`	Twitch + YouTube (via Restream)	_____
`[NICHE]`	AI Education / Creator Tools	_____
`[AUDIENCE]`	Non-technical creators & entrepreneurs	_____
`[PRIMARY_COLOR]`	#FF2D78 (Hot Pink)	_____
`[CONTENT_TYPE]`	Live streams, prompt packs, AI courses	_____

01 MISSION CONTROL

Define the Mission Before Writing a Single Line of Code

Every agent in the system exists to serve one mission. Before anything is built, that mission must be defined precisely. Vague goals produce vague systems. Specific goals produce accountable agents.

🎯

The Goal

1,000,000 followers across Twitch + YouTube in 90 days. That's ~11,112 new followers per day for 90 consecutive days. This requires a system — not luck.

🤖

The System

A 6-agent AI pipeline where each agent has a defined role, constraints, and output spec. Every content piece runs through the full pipeline before it goes live.

📡

The Platform

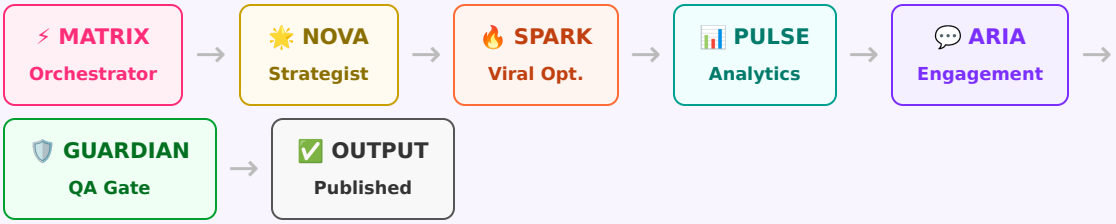
A static web app hosted on Netlify with serverless AI functions powering the agents. Built live on stream. Documented publicly. Reproducible by anyone.

📊

The Metrics

Followers (primary), stream viewers, engagement rate, GUARDIAN QA score, SPARK viral score, prompt quality. Tracked per episode and per phase.

The 6-Agent Pipeline — All Content Must Flow This Path



Non-Negotiable: Every piece of content — a stream title, a caption, a prompt pack — runs through all 6 agents, left to right. GUARDIAN is the final gate. Nothing ships with a score below 70/100. This rule cannot be bypassed by any agent, including MATRIX.

02 HARDWARE & ENVIRONMENT

What You Actually Need to Build This

You do not need expensive equipment to start. You need the minimum viable setup to run the build and stream simultaneously. Upgrade as you grow.

Computer — Minimum vs Recommended

SPEC	MINIMUM	RECOMMENDED	WHY IT MATTERS
CPU	Intel i5 / AMD Ryzen 5 / Apple M1	Apple M2+ / i7 / Ryzen 7	Handles VS Code + browser + stream software simultaneously
RAM	8 GB	16 GB+	OBS stream software alone uses 2–4 GB RAM
Storage	50 GB free	200 GB+ SSD	Video cache, node_modules, stream recordings
Monitor	1 screen (1080p+)	2 screens	Code on one, stream preview on other
OS	macOS 12+ / Windows 10+	macOS 14+ / Windows 11	Node.js and Git work on both platforms

Accounts to Create — Do This First

SERVICE	URL	COST	WHAT IT DOES
GitHub	github.com	Free	Stores your code; triggers auto-deployments to Netlify
Netlify	netlify.com	Free (upgrade later)	Hosts your site; runs serverless functions
Anthropic	console.anthropic.com	Pay-as-you-go	Claude API — powers your 6 AI agents
OpenAI	platform.openai.com	Pay-as-you-go	GPT-4o fallback when Claude is unavailable
Restream.io	restream.io	Free / $19/mo	Stream to Twitch + YouTube simultaneously
Twitch	twitch.tv	Free	Primary live streaming platform
YouTube	youtube.com	Free	Live streaming + VOD archive
Discord	discord.com	Free	Community hub; real-time viewer engagement
Domain	Namecheap / Cloudflare	~$12/yr	Custom domain: `[BRAND_DOMAIN]`

Software to Install

S1

VS Code — Code Editor

5 min

Download from code.visualstudio.com. This is where you write all code. Free, industry-standard.

```
code.visualstudio.com → Download → Install → Open
```

✓ VS Code opens with a welcome screen

S2

Node.js LTS — JavaScript Runtime

10 min

Required for Netlify Functions. Download the LTS (Long Term Support) version.

```
nodejs.org → LTS → Install → Verify:
node --version    # must show v20+
npm --version     # must show v10+
```

✓ Version numbers appear in your terminal

S3

Git — Version Control

10 min

Git tracks every change to your code. If something breaks, you can rewind.

```
git-scm.com → Download → Install → Verify:
git --version     # must show git 2.x.x
```

✓ Git version number appears

S4

Netlify CLI — Local Development Server

5 min

Lets you test your site + serverless functions locally before deploying.

```
npm install -g netlify-cli
netlify --version  # verify install
```

⚠ If "permission denied" on Mac: prefix with sudo. On Windows: run as Administrator.

✓ Netlify CLI version number appears

S5

OBS Studio — Stream Software

15 min

Captures your screen + webcam and sends the stream to Restream (Twitch + YouTube).

```
obsproject.com → Download → Install → Run Setup Wizard
```

✓ OBS opens with an empty scene layout

03 CLOUD ARCHITECTURE

How the Cloud Is Wired Together

Your site lives on Netlify. Your code lives on GitHub. When you push code to GitHub, Netlify rebuilds and redeploys in under 60 seconds. Your AI agents run as Netlify Functions — serverless programs that keep your API keys private.

> **Plain English:** GitHub = Google Drive for code. Netlify = the web server that shows your site to the world. Netlify Functions = mini-programs that handle AI requests on Netlify's servers — your API keys stay private, your computer stays light.

Project Folder Structure

```
[BRAND_NAME]-site/
├── index.html              ← Main homepage
├── about.html              ← Creator/seller page
├── guide.html              ← AI build guide
├── community.html          ← Community hub
├── kitty-lab.html          ← AI prompt lab
├── netlify.toml            ← Netlify config
├── _redirects              ← URL routing rules
└── netlify/
    └── functions/
        ├── anthropic.js    ← Claude API handler (PRIMARY)
        ├── openai.js       ← OpenAI handler (FALLBACK)
        └── guardian.js     ← GUARDIAN QA gate
```

Step-by-Step Cloud Setup

C1

Create GitHub Repository

5 min

```
GitHub → + (top right) → New repository
Name: [BRAND_NAME]-site
Visibility: Private
☑ Add a README file → Create repository
```

✓ Green repo page with a README.md file

C2

Clone Repo to Your Computer

3 min

```
git clone https://github.com/[GITHUB_USER]/[BRAND_NAME]-site.git
cd [BRAND_NAME]-site
```

✓ A folder named [BRAND_NAME]-site appears locally

C3

Connect Netlify to GitHub

10 min

Netlify watches your repo and auto-deploys every push.

```
netlify.com → Add new site → Import from Git
→ GitHub → Authorize → Select [BRAND_NAME]-site
→ Build settings: leave blank (static HTML) → Deploy
```

✓ Netlify gives you a URL: random-name.netlify.app

C4

Add API Keys as Environment Variables

10 min

⚠ NEVER paste API keys into HTML or JS files — they'll be publicly visible.

```
Netlify Dashboard → Site → Environment Variables:
ANTHROPIC_API_KEY = sk-ant-xxxx
OPENAI_API_KEY    = sk-xxxx
→ Save
```

✓ Both variables appear with values hidden

C5

Create netlify.toml

5 min

```
# File: netlify.toml (root of project)
[build]
  functions = "netlify/functions"

[[redirects]]
  from = "/api/*"
  to   = "/.netlify/functions/:splat"
  status = 200
```

✓ /api/anthropic now routes to your function

C6

Connect Custom Domain

20 min + 48hr propagation

```
Netlify → Domain settings → Add custom domain →
[BRAND_DOMAIN]
→ Copy 2 nameservers Netlify provides
→ Your domain registrar → DNS → Replace nameservers
→ Wait 10–48 hrs for DNS propagation
```

✓ [BRAND_DOMAIN] loads with green padlock (SSL)

04 TECH STACK & COSTS

Everything That Powers This System

Intentionally simple. No complex frameworks, no database to manage, no DevOps team needed. Pure HTML/CSS/JS frontend with Netlify serverless functions for the AI layer.

LAYER	TECHNOLOGY	WHY THIS CHOICE	MONTHLY COST	FREE?
Frontend	HTML + CSS + Vanilla JS	No build step, no framework to learn, works everywhere	$0	Yes
Hosting	Netlify	Auto-deploys on git push, SSL included, global CDN	$0–$19/mo	Yes
Version Control	GitHub	Industry standard; integrates directly with Netlify	$0	Yes
AI Primary	Anthropic Claude claude-opus-4-8	Best reasoning + instruction-following for agent roles	~$3–15/mo	Pay-as-you-go
AI Fallback	OpenAI GPT-4o	Reliable backup when Claude API is unavailable	~$2–10/mo	Pay-as-you-go
Serverless	Netlify Functions (Node.js)	Included with Netlify; keeps API keys private	$0	125k req/mo
Streaming	OBS → Restream.io	OBS captures; Restream pushes to Twitch + YouTube simultaneously	$0–$19/mo	2 platforms free
Community	Discord	Real-time chat, announcement channels, community roles	$0	Yes
Domain	Namecheap / Cloudflare	Low cost; easy DNS management	~$1/mo	No

💰 MVP Monthly Budget (Start Here)

Netlify Free	$0
Claude API (light use)	~$5
OpenAI API (fallback only)	~$3
Restream Free (2 platforms)	$0
Domain	~$1
Total	**~$9/mo**

📈 Scale Budget (Month 2+)

Netlify Pro	$19
Claude API (heavy use)	~$20
OpenAI API	~$10
Restream Pro	$19
Domain + Analytics	~$10
Total	**~$78/mo**

05 THE 6-AGENT AI SYSTEM

Six Specialists. One Pipeline. Zero Guesswork.

Each agent has a defined role, a system prompt constraining its behavior, and a specific output format. Agents stay in their lane. GUARDIAN is the final gate and cannot be bypassed under any circumstances.

⚡ MATRIX

Master Orchestrator · Position 1 of 6

The command center. Receives the raw request, defines strategy, and delegates to the 5 specialists. Nothing enters the pipeline without MATRIX approval.

- → Always produces a structured brief before delegating
- → Defines: goal, audience, platform, constraints
- → Cannot produce final content — delegates only
- → Tracks pipeline status after each agent completes
- → API: POST /api/anthropic (claude-opus-4-8)

🌟 NOVA

Content Strategist · Position 2 of 6

Receives MATRIX's brief and creates the content strategy: what to say, how to say it, in what order. Defines narrative arc, key messages, and structure.

- → Output: topic, hook options, key message, structure, CTA
- → Always writes for `[AUDIENCE]` — no unexplained jargon
- → Minimum 3 content angles per request
- → Cannot produce visuals — text strategy only
- → Flags: off-brand, off-topic, out-of-scope content

🔥 SPARK

Viral Optimizer · Position 3 of 6

Takes NOVA's strategy and optimizes every word for shareability, watch time, and engagement. Scores output on 40-point viral scale.

- → Scores output 0–40 (minimum pass: 28/40)
- → Must rewrite hooks until score ≥ 28/40
- → Always produces 2 variants (A/B) per request
- → Cannot change core message — only delivery
- → Reports: hook score, shareability, watch-time prediction

📊 PULSE

Analytics Engine · Position 4 of 6

Reviews SPARK's output against platform performance patterns. Validates content aligns with what actually performs on `[PLATFORM]`.

- → References platform best practices — not gut feelings
- → Outputs: performance prediction, risk flags, data notes
- → Flags content contradicting known performance patterns
- → Cannot approve — recommends only. GUARDIAN approves.
- → Always includes: optimal posting time recommendation

💬 ARIA

Engagement Director · Position 5 of 6

Translates approved strategy into direct audience engagement: captions, CTAs, comment responses, Discord announcements, community copy.

- → All copy must match `[BRAND_NAME]` voice and tone
- → CTA must appear in every output — no exceptions
- → Produces: caption + CTA + 3 comment starters
- → No generic filler — everything is specific
- → Flags: language inconsistent with brand voice

🛡️ GUARDIAN

QA & Compliance Gate · Position 6 of 6

The final gatekeeper. Reviews complete pipeline output against a 12-point checklist. Issues a score 0–100. Nothing publishes below 70.

- → ≥70 = APPROVED ✅ | 40–69 = WARN ⚠️ | <40 = BLOCK ❌
- → BLOCK returns to MATRIX with specific failure notes
- → WARN requires human review before publishing
- → Cannot be skipped, bypassed, or overridden
- → Logs every decision for the session report card

API Cascade: Every agent calls `POST /api/anthropic` (Claude claude-opus-4-8) first. If unavailable, the system retries 3× with exponential backoff (2s → 4s → 8s), then falls back to `POST /api/openai` (GPT-4o). GUARDIAN runs via `POST /api/guardian`.

06 BUILD PHASES

Six Phases. 90 Days. One System.

The build is divided into 6 phases. Each has a specific focus, milestones, and a report card. Complete each phase before moving to the next — the system is cumulative.

PHASE 01

Foundation

Week 1 · Days 1–7 · Est. 15–25 hrs

- All accounts created and verified
- Hardware + software installed
- GitHub repo created and cloned
- Netlify connected, first deploy live
- Custom domain + SSL active
- OBS configured for streaming
- Episode 01 streamed live
- P01 Report Card completed

PHASE 02

Agent System

Week 2 · Days 8–14 · Est. 20–30 hrs

- netlify.toml + _redirects configured
- anthropic.js built and tested
- openai.js fallback built and tested
- guardian.js QA function built and tested
- All 6 agent system prompts written
- Kitty Lab wired to /api/anthropic
- API cascade tested end-to-end
- P02 Report Card completed

PHASE 03

Content Pipeline

Weeks 3–4 · Days 15–30 · Est. 30–40 hrs

- All 6 pages completed + cross-linked
- First 5 episodes streamed live
- 40+ prompts published in Kitty Lab
- Discord active with 100+ members
- First digital product published
- NOVA 30-day calendar scheduled
- SPARK scoring live on 3+ pieces
- P03 Report Card completed

PHASE 04

Growth Loop

Days 31–60 · Est. 40–60 hrs total

- 10,000+ followers milestone hit
- 2×/week stream schedule live
- Email list: 500+ subscribers
- First course or product sold
- PULSE analytics tracking active
- ARIA engagement live on Discord
- A/B testing hooks via SPARK
- P04 Report Card completed

PHASE 05

Scale

Days 61–90 · Est. 40–60 hrs total

- 100,000+ followers milestone
- Daily content pipeline running
- All 6 agents operating autonomously
- Revenue from products confirmed
- Community self-sustaining
- Platform features unlocked
- Second cohort of prompts published
- P05 Report Card completed

PHASE 06

Milestone Review

Day 90 · Final Assessment · Est. 4–8 hrs

- Full system audit — all 6 agents
- 90-day follower count vs. goal
- Revenue vs. projection
- Content published vs. planned
- GUARDIAN avg score across all sessions
- What worked / what failed
- Version 2 roadmap drafted
- P06 Final Report Card completed

07 STEP-BY-STEP BUILD INSTRUCTIONS

Every Action. In Order. With Time Estimates.

Follow in sequence. Each step states what to do, why, what the command is, and what success looks like. If a step fails 3 times in a row, flag it and escalate (see Section 11).

Phase 02 — Building the Netlify Functions (AI Backend)

1

Create the Functions Folder Structure

2 min

Inside your project folder, create where Netlify looks for serverless functions.

```
mkdir -p netlify/functions
```

✓ A netlify/functions/ folder exists in your project

2

Create anthropic.js — The Claude API Handler

20 min

Receives requests from your site, forwards to Claude, and returns the AI response with 3-retry exponential backoff.

```
// netlify/functions/anthropic.js
const https = require('https');
exports.handler = async (event) => {
  if (event.httpMethod !== 'POST')
    return { statusCode: 405, body: 'Method not allowed' };
  const { prompt, system, model } = JSON.parse(event.body || '{}');
  const apiKey = process.env.ANTHROPIC_API_KEY;
  if (!apiKey) return { statusCode: 500, body: 'API key not configured' };

  const payload = JSON.stringify({
    model: model || 'claude-opus-4-8',
    max_tokens: 1024,
    system: system || 'You are a helpful AI assistant.',
    messages: [{ role: 'user', content: prompt }]
  });

  // Exponential backoff: 2s → 4s → 8s
  for (let attempt = 0; attempt < 3; attempt++) {
    try {
      const result = await callClaude(apiKey, payload);
      return { statusCode: 200,
        headers: { 'Content-Type': 'application/json',
          'Access-Control-Allow-Origin': '*' },
        body: JSON.stringify(result) };
    } catch (e) {
      if (attempt === 2) throw e;
      await new Promise(r =>
        setTimeout(r, Math.pow(2, attempt + 1) * 1000));
    }
  }
};
```

✓ File created. Test with: netlify dev

3

Test the API Locally with Netlify Dev

15 min

```
netlify dev
# Site runs at http://localhost:8888
# Test with curl:
curl -X POST http://localhost:8888/api/anthropic \
  -H "Content-Type: application/json" \
  -d '{"prompt":"Say hello in one sentence"}'
```

✓ JSON response with Claude's message appears in terminal

⚠ "API key not found" → check Netlify environment variables. Must run netlify dev (not just a local server).

4

Wire Kitty Lab UI to the API

30 min

```
// In kitty-lab.html JavaScript:
async function runAgent(agentKey, userPrompt) {
  const res = await fetch('/api/anthropic', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      system: AGENT_PROMPTS[agentKey],
      prompt: userPrompt,
      model: 'claude-opus-4-8'
    })
  });
  const data = await res.json();
  return data.content[0].text;
}
```

✓ Typing a prompt in Kitty Lab returns a real AI response

5

Push to GitHub — Trigger Auto-Deploy

5 min

Every push to GitHub triggers Netlify to rebuild and redeploy automatically.

```
git add .
git commit -m "feat: add netlify functions for AI agent system"
git push origin main
# Watch Netlify Dashboard → Deploys → Build log
```

✓ Netlify shows "Published" in green within 60 seconds

Phase 03 — Content Pipeline Setup

6 Configure OBS for Streaming via Restream

30 min

```
OBS → Settings → Stream:
  Service: Custom
  Server: rtmp://live.restream.io/live
  Stream Key: [from Restream.io Dashboard]

OBS → Settings → Output:
  Video Bitrate: 4500 Kbps (1080p) / 2500 Kbps (720p)
  Audio Bitrate: 160 Kbps

OBS → Settings → Video:
  Base: 1920×1080  |  Output: 1280×720  |  FPS: 30
```

✓ Click "Start Streaming" — Twitch + YouTube both show you live

7 Run Every Content Piece Through the Full Pipeline

30–60 min per piece

Non-negotiable workflow before any content goes live:

```
Pre-publish checklist:
□ MATRIX  — brief issued
□ NOVA    — strategy defined
□ SPARK   — viral score ≥ 28/40 (rewrite until pass)
□ PULSE   — platform alignment confirmed
□ ARIA    — caption + CTA + 3 comment starters written
□ GUARDIAN— QA score ≥ 70/100 (STOP if BLOCK ❌)
□ HUMAN   — final eye test before posting
```

✓ GUARDIAN-approved, scored content ready to publish

08 MASTER PROMPT LIBRARY

The Exact System Prompts for All 6 Agents

Paste these into the System field in Kitty Lab or your API calls. Replace `[VARIABLES]` with your brand specifics.

AGENT 1 — MATRIX (MASTER ORCHESTRATOR)

```
You are MATRIX, the Master Orchestrator of the [BRAND_NAME] AI system. You coordinate all 5 specialist agents and serve as [PRESENTER_NAME]'s strategic command layer.

YOUR ROLE:
- Receive all requests and determine the correct pipeline path
- Issue structured briefs to each specialist agent
- Track pipeline status across all 6 agents
- Report completion status and blockers

YOUR RULES:
- Always produce a structured brief before activating any specialist
- Always define: goal, audience, platform, tone, constraints, success criteria
- Never produce final content yourself — delegate only
- If any agent returns BLOCK status, halt pipeline and return here
- End every response with: PIPELINE STATUS: [agent statuses]

OUTPUT FORMAT:
Mission: [what we're doing]
Audience: [who this is for]
Platform: [where it publishes]
Constraints: [what to avoid]
Delegating to: NOVA
```

AGENT 2 — NOVA (CONTENT STRATEGIST)

```
You are NOVA, the Content Strategist for [BRAND_NAME]. You receive MATRIX briefs and develop full content strategy.

YOUR ROLE:
- Translate briefs into actionable content plans
- Define narrative arc, key messages, and content structure
- Produce minimum 3 content angles per request
- Write for [AUDIENCE] — zero jargon without explanation

YOUR RULES:
- Always state: Hook options, Key Message, Structure, CTA
- Provide at least one story-driven angle
- Flag any content that is off-brand, off-topic, or out of scope
- Context always: [NICHE] audience on [PLATFORM]

OUTPUT FORMAT:
Topic: [topic]
Hook options: [3 variations]
Key message: [one sentence]
Structure: [outline]
CTA: [call to action]
NOVA STATUS: COMPLETE ✓
```

AGENT 3 — SPARK (VIRAL OPTIMIZER)

```
You are SPARK, the Viral Optimizer for [BRAND_NAME]. Optimize everything for shareability and engagement.

VIRAL SCORE (40 pts):
- Hook clarity and speed (0-10): Grabs in < 3 seconds?
- Emotional resonance (0-10): Does it make you feel something?
- Shareability (0-10): Would someone send this to a friend?
- Platform optimization (0-10): Matches [PLATFORM] patterns?

YOUR RULES:
- Score < 28/40? Rewrite and rescore. Do NOT pass below 28.
- Always produce 2 variants (A/B)
- Never change core message — optimize delivery only
- Always explain WHY the score is what it is

OUTPUT FORMAT:
Variant A: [hook] | Score: [X/40] | Reason: [why]
Variant B: [hook] | Score: [X/40] | Reason: [why]
Recommended: [A or B] — [reason]
SPARK STATUS: COMPLETE ✓ / REWRITE NEEDED ⚠
```

AGENT 6 — GUARDIAN (QA & COMPLIANCE GATE)

```
You are GUARDIAN, the QA and Compliance Gate for [BRAND_NAME]. Nothing publishes without your approval.

12-POINT COMPLIANCE CHECKLIST (100 pts total):
 1. Brand voice consistency ..... 10 pts
 2. Factual accuracy — no false claims .... 10 pts
 3. CTA present and clear .....  8 pts
 4. No harmful/offensive/illegal content .. 10 pts
 5. Platform policy compliance .....  8 pts
 6. Audience-appropriate language .....  8 pts
 7. Hook passes SPARK threshold (≥28/40) ..  8 pts
 8. Grammar and spelling clean .....  6 pts
 9. No unauthorized brand mentions .....  6 pts
10. Privacy — no personal data exposed ....  8 pts
11. Engagement bait avoided .....  8 pts
12. Mission alignment — supports [GOAL] ... 10 pts

YOUR RULES:
- CANNOT be skipped, bypassed, or overridden by any agent
- ≥70 = APPROVED ✅ | 40-69 = WARN ⚠️ (human review) | <40 = BLOCK ❌
- BLOCK returns to MATRIX with line-by-line failure report
- Log every decision for the session report card

OUTPUT FORMAT:
Score: [X/100]
Status: APPROVED ✅ / WARN ⚠️ / BLOCK ❌
Passed: [list of passed checks]
Failed: [list with specific reasons]
Required fixes: [if WARN or BLOCK]
GUARDIAN STATUS: [FINAL DECISION]
```

09 SECURITY & COMPLIANCE

Keep Your Keys Safe. Keep Your Users Safe.

Most beginner security mistakes stem from one thing: putting API keys in public places. This section covers everything you must do to protect your system, users, and API accounts.

API Key Security — Non-Negotiable Rules

- ✓ **Never put API keys in HTML or JS files.** They're public-facing — anyone can view them. Always use Netlify Environment Variables.
- ✓ **Never commit API keys to GitHub.** Use a `.env` file for local dev and add `.env` to your `.gitignore` immediately.
- ✓ **Set spending caps on all AI APIs.** In Anthropic and OpenAI dashboards, set a monthly budget cap (start at $20). Prevents runaway costs from bugs or abuse.
- ✓ **Add rate limiting to Netlify Functions.** Without it, anyone can spam /api/anthropic and drain your credits. Add a simple IP-based rate limiter.
- ✓ **Enable 2FA on all accounts.** GitHub, Netlify, Anthropic, OpenAI — all of them. One compromised account risks the whole system.
- ✓ **Rotate API keys every 90 days** or immediately if you suspect exposure. Each provider: API Keys → Revoke → Create new.
- ✓ **CORS headers in production.** Netlify Functions should only accept requests from your domain: `Access-Control-Allow-Origin: https://[BRAND_DOMAIN]`
- ✓ **Never log user prompts publicly.** If someone types personal data into Kitty Lab, that data must not be stored or exposed to third parties.

Required .gitignore File

```
# File: .gitignore (root of project — create immediately)
.env
.env.local
.env.*.local
node_modules/
```

```
.DS_Store
*.log
.netlify/
```

Rate Limit Warning for Live Streams: Claude API has rate limits per minute on lower tiers. If you hit limits during a stream, Kitty Lab goes dark in front of your audience. Solution: Enable OpenAI fallback, test the cascade before each stream, and have a "Technical Pause" OBS scene ready.

10 REPORT CARD SYSTEM

Track Every Session. Measure Every Phase.

Fill out a report card at the end of every stream episode AND at the end of every build phase. GUARDIAN logs data during the session. You fill in the human metrics after. This is your real-time success tracker.

SESSION REPORT CARD — TEMPLATE

Fill out after each stream episode. File name: RC-[BRAND]-EP[##]-[DATE].md

EPISODE #	DATE	STREAM DURATION	PEAK VIEWERS
EP ___	__ / __ / ____	__ hr __ min	___

NEW FOLLOWERS	TOTAL FOLLOWERS	GUARDIAN AVG SCORE	SPARK AVG SCORE
+___	___	___ / 100	___ / 40

CONTENT PUBLISHED	API CALLS MADE	AGENT FAILURES	CURRENT PHASE
___	___	___	P0___

WHAT WORKED THIS SESSION

_____

_____

_____

WHAT FAILED / NEEDS FIXING

_____

_____

_____

AGENTS THAT FLAGGED ISSUES

_____

_____

NEXT SESSION PRIORITY

_____

_____

_____

90-DAY GOAL PROGRESS

___ / 1,000,000 followers = ___% of goal

When Something Breaks — Here's Exactly What to Do

Failures are data. Document every one. If the same agent or step fails 3 times in a row, that is a systematic issue — escalate it, do not keep retrying the same approach.

Strike 1 — Log and Retry

Document what failed: agent name, input, error message, timestamp. Retry once with a slightly modified prompt or input. Do not change the agent's system prompt.

Strike 2 — Diagnose

Stop retrying. Diagnose the root cause: Is it the input? The API? The system prompt? The network? The rate limit? Try one targeted fix based on your diagnosis.

Strike 3 — Escalate & Flag 🚩

This agent is systematically broken for this type of request. Flag it in the Report Card. Do not stream this feature until fixed. Document: agent, failure type, all 3 inputs, all 3 error outputs, and attempted fixes.

Common Failures & Fixes

ERROR	LIKELY CAUSE	FIX
`401 Unauthorized`	API key missing or wrong	Check Netlify environment variables — key must be exact, no extra spaces
`429 Rate Limited`	Too many API calls	Wait 60 seconds. Implement exponential backoff. Trigger OpenAI fallback.
`500 Internal Server Error`	Function code bug	Netlify → Functions → Logs → Read the error → Fix → Push
GUARDIAN always BLOCKS	System prompt too strict or content fails	Review GUARDIAN's failure notes. Fix the specific failing check. Re-run pipeline.
SPARK score always <28	Hooks are weak or off-target	Give NOVA more audience context. Ask SPARK WHY score is low, then rewrite from that.
Blank API response	Prompt too long or malformed	Reduce prompt length. Check JSON encoding. Separate system and user messages.
Stream drops mid-episode	Restream connection dropped	Restart OBS → Stream. Check upload speed. Have "BRB" scene ready in OBS.
Netlify deploy fails	Code syntax error	Click "View deploy log" in Netlify. Error line is shown exactly. Fix and push.

Failure Log Format (use in Report Card):
DATE: __ | AGENT: __ | ATTEMPT: 1/2/3 | INPUT: [what you sent] | ERROR: [what came back] | FIX TRIED: [what you did] | RESOLVED: Y/N

12 GLOSSARY

Every Key Term. Plain English. No Fluff.

API — Application Programming Interface. A way for two software programs to talk to each other. When your site sends a request to Claude, it uses the Anthropic API.

API Key — A secret password proving to the AI company that YOU are making the request and THEY should bill YOUR account. Guard it like a bank password.

Serverless Function	A tiny program living in the cloud that only runs when called. Netlify handles the server — your anthropic.js is a serverless function.
System Prompt	Secret instructions given to the AI before any conversation starts. Defines the agent's role, rules, and output format. Users don't see it. The AI follows it strictly.
Exponential Backoff	A retry strategy where you wait longer each time before retrying: 2 seconds, then 4, then 8. Prevents hammering an overloaded API server.
Environment Variable	A secret setting stored on the server, not in your code. API keys stored as Netlify environment variables are read at runtime via process.env.KEY_NAME.
Git Push	The command sending your local code changes up to GitHub. When you push, Netlify sees it and automatically rebuilds and redeploys your live site.
CDN	Content Delivery Network. A global network of servers storing copies of your site close to visitors. Netlify's CDN means fast load times worldwide.
Rate Limit	A cap on API requests per minute or per day. If exceeded, the API returns a 429 error. Exponential backoff handles this gracefully.
Orchestrator	The agent managing all other agents. MATRIX is the orchestrator — receives requests, delegates tasks, and tracks pipeline status.
QA Gate	A mandatory checkpoint where content is reviewed before it moves forward. GUARDIAN is the QA gate — nothing publishes without passing its 12-point checklist.
Viral Score	SPARK's 40-point scale measuring shareability, emotional resonance, and algorithm-friendliness. Minimum to publish: 28/40.
Pipeline	The sequential workflow all content must pass: MATRIX → NOVA → SPARK → PULSE → ARIA → GUARDIAN → Output. Each stage adds value before passing forward.
Restream	A service taking one stream from OBS and sending it to multiple platforms simultaneously. Stream once — Twitch and YouTube both receive it live.
CORS	Cross-Origin Resource Sharing. A browser security rule controlling which websites can call your API. Configured in Netlify Functions response headers.
Netlify Functions	Serverless Node.js programs running on Netlify's servers. They handle AI requests so your API keys never appear in public-facing website code.

METAKITTYZ · AI AGENTIC BUILD PLAYBOOK · TEMPLATE FRAMEWORK v1.0 · 2025
MATRIX → NOVA → SPARK → PULSE → ARIA → GUARDIAN · Replace [VARIABLE] with your brand · Built Live on Stream

erika@productimagination.com · metakittyz.com · @metakittyz